설비 고장을 예측하는 AI 데이터 분석

이상치 처리

IQR 이상치 처리와 중복 데이터

posco

CONTENTS

목차

01 IQR 이상치 탐색 — IQR이란 무엇인가, 이상치 경계 공식

02 이상치 처리와 중복 — 처리 방법, 중복 데이터 정리

03 직접 분석 실습 — IQR 흐름과 중복 정리까지 손에 익히기

posco

01

IQR 이상치 탐색

IQR 계산과 이상치 경계 — 사분위 범위로 자동 탐색

posco

다이캐스팅 주조 공정 이해 — 도메인 노트

데이터를 읽기 전, 이 설비가 무엇을 재는지 먼저 이해

① 공정

다이캐스팅은 녹인 금속을 고압으로 금형에 주입해 부품을 성형하는 주조 방식. 한 번의 주입을 샷이라 부른다.

② 핵심 변수

실린더압력은 금속을 밀어넣는 유압, 사이클타임은 한 샷에 걸린 시간, 비스킷두께는 잔류 금속, 형체력은 금형을 무는 힘을 뜻한다.

③ 정상과 이상

정상 샷은 사이클타임 20~35초에 압력이 일정 범위를 유지한다. 잼이나 미충전이 생기면 사이클타임이 급증하고 압력이 벗어난다.

④ 왜 이 분석

대부분이 정상이고 이상은 드물게 튄다. 그래서 IQR로 가운데 절반을 기준 삼아 벗어난 샷을 이상 후보로 골라내고 보정한다.

⑤ 캡스톤 — 용접

용접은 통전전류가 정상 범위를 벗어나면 접합 불량이 된다. 주조에서 익힌 IQR 방법을 다른 현장에도 그대로 적용해 본다.

posco

IQR이란 무엇인가

사분위 범위 — 가운데 절반이 차지하는 폭, 양 끝 이상치에 안 흔들림

01 · 정의

Inter-Quartile Range

IQR =
Q3 – Q1

Q3에서 Q1을 뺀 값, 새 계산 없이 뺄셈 한 번

02 · 온도 예시

Q3 81.5, Q1 74

81.5 – 74
= 7.5도

단위가 원본과 같아 '가운데 절반이 7.5도 폭'으로 바로 해석

03 · 강점

이상치에 흔들리지 않는 지표

가운데 절반만
집중 관찰

양 끝값을 안 보므로 극단값에 휘둘리지 않음

IQR과 범위의 차이

범위는 양 끝값에 휘둘리지만, IQR은 극단값이 섞여도 안 흔들림

범위

최댓값 – 최솟값

양 끝값만 봄

이상치 하나만 들어와도 결과가 크게 흔들림

250·300 한 점이 그대로 범위 폭으로 반영

IQR

Q3 – Q1, 가운데 절반의 폭

가운데 절반만 봄

극단값이 섞여도 거의 안 흔들림

가운데 50%에는 극단값이 안 들어오므로 안정적

이 안정성이 IQR을 이상치 탐색의 든든한 기준으로 만듦

posco

IQR을 기준으로 쓰는 이유

데이터마다 퍼짐이 다르므로 그 데이터 자신의 IQR을 잣대로 사용

모든 데이터에 똑같은 고정 숫자를 쓸 수 없음

컬럼마다 퍼짐이 다르고, 정상 범위도 제각각

그래서 그 데이터 자신의 퍼짐(IQR)을 잣대로 사용

그 데이터가 보통 얼마나 흩어지는지를 기준으로 삼는 발상

데이터 성격에 맞춰 경계가 자동 조절

넓게 퍼진 데이터는 넉넉하게, 좁게 모인 데이터는 빡빡하게

온도와 다른 센서의 정상 범위가 달라도 같은 규칙으로 처리 가능

posco

컬럼이 많아도 한 번에

수백 컬럼도 같은 규칙으로 한 번에 적용 — 사람이 손볼 일이 없음

01 · 자동화

수동 기준 설정 불필요

컬럼마다 따로
정할 필요 없음

사람이 컬럼마다 기준을 정하지 않아도 됨

02 · 확장성

수백 컬럼도 일괄 적용

SECOM도
같은 규칙으로

수백 컬럼도 같은 규칙으로 한 번에 적용

03 · 견고함

잣대가 망가지지 않음

이상치에
안 흔들림

IQR 자체가 흔들리지 않으므로 잣대가 망가지지 않음

사람의 직관 — 가운데 모인 값을 기준으로 너무 먼 값을 이상하다고 느끼는 것 — 을 수치로 옮긴 도구

posco

이상치 경계 공식

가운데 절반 구간에 IQR의 1.5배 여유 — 그 바깥이 이상치

LOWER · 하한

$$Q1 - 1.5 \times IQR$$

이 값보다 작으면 이상치

UPPER · 상한

$$Q3 + 1.5 \times IQR$$

이 값보다 크면 이상치

박스플롯에서 수염 밖에 찍히던 점이 바로 이 공식으로 잡은 값. 통계에서 가장 널리 쓰이는 표준 공식

posco

우리 데이터에 넣어보기

온도 데이터 — 하한 62.75, 상한 92.75. 250·300·−5가 정확히 이상치

단계	계산	값
IQR	81.5 − 74	**7.5**
하한	74 − 1.5 × 7.5	**62.75**
상한	81.5 + 1.5 × 7.5	**92.75**

> −5는 하한 아래, 250·300은 상한 위 → 모두 이상치. 72·78 같은 정상값은 모두 그 사이

posco

왜 하필 1.5배인가

경험적으로 정한 균형점 — 너무 좁으면 정상값까지, 너무 넓으면 진짜 이상치를 놓침

1배로 좁히면

너무 예민한 경보기

정상값까지 솎아냄

조금만 벗어나도 잡혀 멀쩡한 값이 이상치로 분류

별일 아닌데 자꾸 울리는 셈

3배로 넓히면

너무 둔한 경보기

진짜 이상치 놓침

웬만큼 튀어도 통과해 정말 이상한 값을 놓침

불이 나도 안 울리는 격

절대 법칙이 아니라 잘 들어맞는 균형점 — 통계학자가 경험적으로 정한 값

posco

1.5배는 균형점

정상값 과솎과 진짜 이상치 누락 사이 — 가장 널리 쓰는 표준 배수

01 · 표준

가장 널리 쓰이는 배수

1.5배
균형점

정상값을 과하게 솎지도, 진짜 이상치를 과하게 놓치지도 않는 지점

02 · 극단

더 극단만 잡고 싶을 때

3배 기준도
존재

아주 극단적인 값만 보고 싶을 때 선택

03 · 조절

데이터 성격에 맞춰

조절 가능한
손잡이

1.5는 기본값, 필요에 따라 조절 가능

posco

경계 계산 코드

Q1·Q3을 구해 IQR을 만들고, 1.5배로 하한·상한 산출

CODE

```
Q1 = df["temperature"].quantile(0.25)
Q3 = df["temperature"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
print(lower, upper)
```

결과: 62.75와 92.75

posco

변수 다섯 개만 채우기

Q1·Q3·IQR·lower·upper — 다섯 개만 차례로 채우면 완성

VAR 1

Q1

25% 분위수

VAR 2

Q3

75% 분위수

VAR 3

IQR

Q3 − Q1

VAR 4

lower

Q1 − 1.5×IQR

VAR 5

upper

Q3 + 1.5×IQR

의미가 드러나는 이름이면 나중에 읽기 쉬움 · 컬럼 이름만 바꿔 어디든 적용 가능

이 다섯 줄이 IQR 탐색의 심장

Boolean indexing 다시 보기

체처럼 참인 칸만 통과시키고 거짓은 걸러내는 도구

컬럼에 조건을 걸면 행마다 True/False가 매겨짐 (마스크)

예) df["temperature"] > 92.75 → 참/거짓이 줄줄이

이 마스크를 대괄호에 넣으면 True인 행만 골라냄

df[mask] 형태로 조건에 맞는 행만 필터

마스크가 체처럼 작동

참인 칸만 통과시키고 거짓은 걸러냄

앞서 이미 여러 번 써 본 익숙한 도구

posco

여러 조건 연결 시 주의

괄호로 감싸기 · 기호 연산자(| &) · 영어 단어(and/or) 금지

01 · 괄호

조건마다 감싸기

```
(df["x"] < lower)
| (df["x"] > upper)
```

각 조건을 괄호로 묶지 않으면 우선순위 오류

02 · 기호

또는 = | / 그리고 = &

| · 또는

& · 그리고

세로 막대와 앰퍼샌드를 사용

03 · 금지

영어 단어 사용 금지

and

or

영어 단어를 쓰면 오류 — 반드시 기호로

자주 하는 실수 — 괄호 누락과 영어 단어 사용

posco

조건으로 이상치 골라내기

mask로 이상치 행만, 물결(~)로 정상 행만 — −5·250·300이 이상치

CODE

```
mask = (df["temperature"] < lower) | (df["temperature"] > upper)
print(mask.sum())
print(df[~mask].shape)
```

posco

골라냄은 아직 탐색일 뿐

골라낸 것뿐, 지우거나 바꾼 게 아님 — 다음 단계는 사람의 판단

01 · 현재 상태

원본 데이터는 그대로

지우거나 바꾼 게
아님

골라내기만 했을 뿐 — 데이터에 손대지 않음

02 · 다음 단계

노이즈인지 신호인지 판단

판단은
다음 단계의 일

함부로 곧바로 지우지 않는 신중한 태도

03 · 검산

골라낸 목록을 출력해 확인

목록 확인이
곧 검산

엉뚱한 값이 잡히면 경계·컬럼 지정 오류 가능성

posco

이상치 개수와 비율

True는 1로 취급 — sum이면 개수, mean × 100이면 퍼센트

CODE

```
print(mask.sum())
print(round(mask.mean() * 100, 1))
```

posco

비율을 함께 봐야 하는 이유

처리 전 규모 파악은 수술 전 검사와 같음

01 · 안전

1% 안팎

지워도 무방

처리 안정적

소수의 명백한 노이즈일 가능성

02 · 재검토

20~30%

처리 방법

재검토 필요

무작정 지우기에는 비중이 너무 큼

03 · 의심

너무 높은 비율

경계 설정

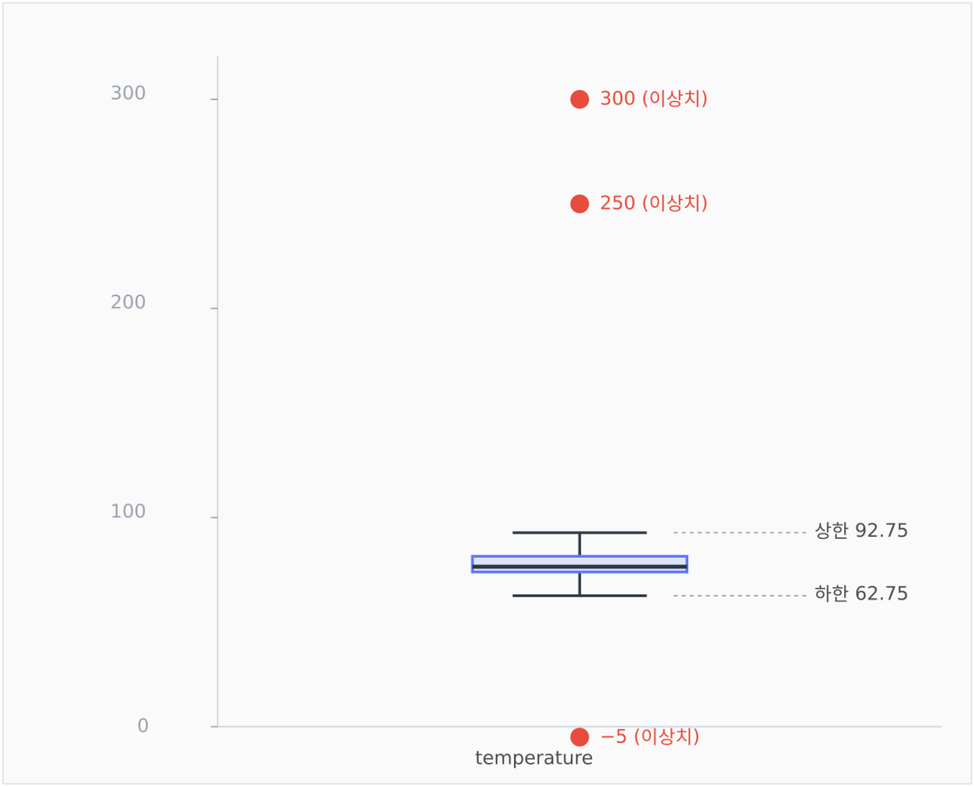
의심 필요

데이터 특성이나 경계 자체가 잘못됐을 가능성

posco

박스플롯으로 이상치 확인

수염 밖 점이 곧 1.5×IQR 경계를 벗어난 값



BOXPLOT 읽는 법

상자 — Q1 74 ~ Q3 81.5

상자는 가운데 절반의 폭 — IQR 그 자체

수염 — 하한 62.75 / 상한 92.75

수염 끝이 곧 계산한 하한·상한과 일치

수염 밖 점 — 250 · 300 · −5

코드로 고른 이상치와 그림의 점이 정확히 일치

그림과 숫자가 정확히 같은 결과를 가리킴

posco

숫자와 그림은 서로를 검증

오늘은 그리기가 아니라 일치를 확인하는 데 집중

mask로 고른 값과 수염 밖 점은 정확히 같아야 함

두 결과가 다르면 코드나 그림 어느 한쪽에 오류

오늘은 그리기가 아닌 일치 확인이 목적

예쁘게 그리는 게 아니라 두 방식이 같은 결과를 가리키는지 확인

박스플롯 그리는 함수는 이후 학습에서 자세히 학습

통계를 몰라도 점 하나로 이상치를 한눈에 보여 주는 강력한 도구

posco

IQR이 놓치는 것

IQR도 만능은 아님 — 세 가지 한계를 알고 쓰기

01 · 단변량

한 컬럼씩만 봄

여러 센서의 결합 이상은 못 잡음

각 컬럼은 정상인데 함께 보면 이상한 경우는 모델 방법이 강함

02 · 분포

모양을 덜 따짐

한쪽 치우친 데이터에 약함

한쪽으로 치우친 데이터는 과대·과소 탐지 가능

03 · 해석

노이즈/신호 구분 못함

통계적 이상만 알려 줌 — 판단은 사람

250이 센서 오류인지 과열인지는 사람이 판단

이후 학습의 모델 기반 방법이 결합 이상에는 더 강함

한계를 알고 강점을 쓰자

강점은 활용하고 한계는 사람의 판단으로 메우기

01 · 역할

똑똑한 조수

후보를 짚어주고
최종 결정은 사람

이상치 후보를 빠르게 찾아주는 자동 조수

02 · 평가

한계가 있다고 약함 아님

약한 도구가
아님

한계를 안다고 도구의 가치가 떨어지지 않음

03 · 출발점

가장 자주 쓰는 기본

든든한
출발점

가장 기본이면서 가장 자주 쓰는 든든한 도구

도구의 강점과 한계를 함께 아는 사람이 도구를 가장 잘 씀

posco

02

이상치 처리와 중복 데이터

제거 · 보정 · 결측 변환과 중복 정리 흐름

이상치 처리 방법 세 가지

어떻게 남기거나 바꿀지 결정 — 무조건 지우기가 아님

STEP 1

제거

이상치 행을 삭제. 명백한 노이즈에 사용

가장 단순하지만 행을 잃음

STEP 2

경계값 보정

상한/하한 값으로 눌러 정돈, 행은 보존

행 살리고 극단값만 정돈

STEP 3

결측치 변환

NaN으로 바꾼 뒤 주변 값으로 채움

가장 부드러운 처리

찾은 이상치를 어떻게 손볼지 결정 — 컬럼마다 다르게도 가능

posco

처리 방법 비교

상황에 따라 골라 쓰기 — 어떤 컬럼은 제거, 어떤 컬럼은 보정

방법	하는 일	언제 쓰는가
제거	행 삭제	**명백한 노이즈, 낮은 비율**
보정	경계값으로 변경	**행 보존하며 극단만 누름**
결측 변환	NaN 후 채움	**주변값으로 자연스럽게 대체**

기계적으로 한 방법만 쓰지 말기 — 값과 데이터 용도에 따라 결정

posco

잊지 말 네 번째 — 보존

노이즈는 정리, 신호는 보존 — 그것이 원칙

01 · 의심 신호

고장 신호로 의심되면

처리하지 않고
보존·표시

극단값의 크기 자체가 고장 정도를 나타냄

02 · 원칙

노이즈 vs 신호

노이즈는 정리
신호는 보존

실제 제조 연구는 사분위수로 찾아 중앙값으로 바꾸는 방식이 흔함

03 · 사후 확인

처리 후 결과 검증

반드시
결과 확인

바꾼 결과가 의도대로 됐는지 항상 점검

처리도 중요하지만 보존이 더 중요한 순간이 있음

이상치 제거하기

물결(~)로 이상치 아닌 행만 남기기 — 새 변수에 담아 원본 보존

CODE

```
df_clean = df[~mask]
print(len(df), len(df_clean))
print(df_clean["temperature"].mean())
```

18행 → 13행, 평균 75.85

posco

제거는 신중하게

한 번 지운 데이터는 못 되돌림 — 명백한 노이즈에만

01 · 안전수칙

새 변수로 분리

결과는 새 변수에 원본 보존

df_clean처럼 새 변수에 담아 원본 데이터 보존

02 · 부수 피해

같은 행의 다른 값도 소실

멀쩡한 진동·압력 값도 함께 사라짐

행을 통째로 없애면 같은 행의 멀쩡한 값도 소실

03 · 적용 조건

제한된 상황에만

명백한 노이즈, 낮은 비율일 때

이 두 조건이 모두 충족될 때만 안전

그 값이 고장 신호였다면 단서를 버리는 것 — 확인 후에만 제거

경계값으로 보정하기

clip — 하한보다 작은 값은 하한으로, 상한보다 큰 값은 상한으로

CODE

```
df["temp_clipped"] = df["temperature"].clip(lower=lower, upper=upper)
print(df["temp_clipped"].min())
print(df["temp_clipped"].max())
print(df["temp_clipped"].mean())
```

최소 62.75 · 최대 92.75 · 평균 78.88

posco

보정의 장점과 주의점

행은 살리고 극단값 크기 정보는 잃음 — 값의 의미를 먼저

장점

행을 잃지 않음

데이터가 아깝거나
행 수가 중요할 때 유용

여러 컬럼을 다룰 때 다른 컬럼의 멀쩡한 값을 지키며 처리 가능

주의점

정보 손실

250과 300이 모두 92.75
크기 정보 사라짐

얼마나 컸는지는 잃고, 경계를 넘었다는 사실만 남음

큰 값 자체가 고장 정도를 나타낸다면 보정보다 보존이 나음

posco

결측치로 바꿔 채우기

mask 함수로 이상치 자리를 NaN으로 바꾸고 중앙값으로 채움

CODE

```
masked = df["temperature"].mask(mask)
print(masked.isna().sum())
med = masked.median()
fixed = masked.fillna(med)
print(fixed.mean())
```

결측 5개 채움 · 평균 75.61 · 함수 mask와 변수 mask 혼동 주의

왜 중앙값으로 채우나

세 처리 중 가장 부드러운 방식 — 주변과 어울리는 값으로 대체

버리지도 경계에 붙이지도 않고 주변과 어울리는 값으로 대체

이상치를 자연스럽게 정상 분포 속에 흡수

평균은 이상치에 오염됐을 수 있어 중앙값으로 채움

이상치에 흔들리지 않는 안정적 대표값

실제 반도체 공정 연구에서도 사분위수 + 중앙값 대체 사용

현장 검증된 표준 흐름

시계열은 앞뒤 값으로 채우기도 — 이후 학습에서 다룸

처리 전후 통계 비교

평균이 중앙값(76.5) 근처로 모임 — 처리 성공의 신호

구분	평균	중앙값
처리 전	115.6	76.5
제거 후	75.85	76.5
보정 후	78.88	76.5
결측 채움	75.61	76.5

처리 전 평균 115는 중앙값과 40도 벌어졌으나 처리 후 모두 중앙값 근처로 수렴

posco

비교에서 함께 물을 것

통계가 안정됐다고 무조건 좋은 처리는 아님

01 · 성공 신호

평균과 중앙값이 가까워짐

처리 성공
신호

이상치에 흔들리던 평균이 안정화

02 · 함정

신호성 이상치였다면

정보 손실
가능성

고장 신호를 지워 깔끔해진 것이라면 정보를 잃은 것

03 · 습관

무언가 바꿨으면

반드시
바뀐 결과 확인

전후 비교 습관은 모든 데이터 가공에 동일하게 적용

전후 비교는 가공 작업의 마지막 안전망

중복 데이터란 무엇인가

이상치가 값 하나의 문제라면, 중복은 행 하나가 통째로 반복되는 문제

01 · 정의

똑같은 내용의 행

두 번 이상
반복 입력

같은 내용의 행이 두 번 이상 등장하는 상태

02 · 영향

같은 값이 두 번 세어짐

통계를
왜곡

평균과 빈도가 부풀려져 모델 학습도 편향

03 · 처리

새 정보 없음

발견하면
대체로 정리

하나만 남기고 정리하는 게 기본

샘플 데이터에 중복 행 2개를 일부러 넣어 둠

중복은 왜 문제인가

설문지에 같은 답이 두 번 들어간 것 — 결과가 한쪽으로 치우침

설문지에 같은 사람이 똑같은 답을 두 번 낸 것과 같음

한 명의 의견이 두 표로 반영되는 비유

그 의견이 두 번 반영되어 결과가 한쪽으로 치우침

평균·빈도·모델 학습 모두 영향

완전히 같은 중복은 새 정보가 없으므로 하나만 남기고 정리

제거가 거의 유일한 처리 방법

다만 무작정 지우기 전, 잘못 들어온 것인지 우연인지 한 번 생각하기

posco

제조 데이터의 중복 원인

수집·병합·가공 — 데이터를 옮기는 모든 단계에서 발생

01 · 수집 중복

통신·시스템 재시작

같은 값이
두 번 기록

통신 재전송이나 프로그램 재시작으로 동일 값 두 번 입력

02 · 파일 병합

겹치는 구간

날짜·설비별 파일을
합칠 때

1월·2월 데이터를 합치는데 둘 다 1월 말이 들어 있으면 중복

03 · 가공 실수

조건 오류, 복붙

같은 행
반복

추출 조건 오류나 복사 붙여넣기로 같은 행 반복

같은 파일을 실수로 두 번 이어 붙이면 전체가 두 배가 됨

posco

중복은 수집 문제의 단서

지우는 데서 그치지 말고 왜 생겼는지 보기

중복은 데이터를 모으고 합치고 옮기는 과정에서 발생

데이터 자체의 문제라기보다 처리 과정의 문제

중복이 있다는 것은 수집 과정에 문제가 있었다는 단서

원인을 짚으면 같은 문제 재발 방지

파일 합칠 때는 합치기 전후 행 수를 비교해 점검

합친 데이터의 행 수가 두 원본의 합과 같은지 확인

외부에서 받은 데이터나 병합한 데이터는 중복을 더 의심

posco

완전 중복과 부분 중복

완전은 모든 컬럼 동일, 부분은 핵심 컬럼만 동일

완전 중복

모든 컬럼 값이 같음

하나만 남기면 됨

설비·회차·온도·진동·압력이 전부 같으면 완전 중복

샘플 데이터에 넣어 둔 케이스

부분 중복

핵심 컬럼만 같음

기준 컬럼 정의 필요

설비·회차는 같은데 온도만 미세 차이가 있다면 부분 중복

같은 회차 측정은 하나여야 한다는 관점

무엇이 같으면 같은 기록인지 사람이 정해 줘야 함

posco

부분 중복은 더 까다롭다

무엇을 기준으로 같은 기록인지 사람이 정해야 함

어떤 컬럼을 기준으로 같은 기록인지 사람이 정해 줘야 함

설비 이해가 있어야 기준 컬럼을 정할 수 있음

작은 데이터는 완전 중복이 대부분이라 단순

학습용 데이터에서는 부분 중복을 만나기 쉽지 않음

실제 제조 데이터는 부분 중복이 더 자주 문제

같은 설비·같은 시각 기록이 소수점 끝자리만 다르게 두 번 들어오는 일이 흔함

기준 컬럼을 정하는 데에도 설비 이해가 필요

duplicated()로 중복 찾기

행마다 앞에 똑같은 게 있었는지 True/False로 표시 — 처음은 False, 두 번째부터 True

CODE

```
print(df.duplicated())
print(df[df.duplicated()])
```

duplicated 결과 활용

이상치 mask와 똑 닮은 True/False — 그대로 출력하면 길어 보기 어려움

이상치 mask와 똑 닮은 True/False 결과

이상치 탐색에서 익힌 마스크 개념 그대로 적용

그대로 출력하면 길어서 보기 어려움

행 수가 많으면 화면을 다 못 채움

행을 걸러 중복만 보기 또는 sum으로 개수만 확인

df[df.duplicated()] · df.duplicated().sum()

일부 컬럼만 기준으로 하려면 subset 옵션 — 곧 다룸

중복 개수 확인하기

True가 1로 더해져 중복 개수 — 우리 데이터는 2개

CODE

```
print(df.duplicated().sum())
print(len(df))
```

이상치 개수를 셀 때와 완전히 같은 방법

posco

개수 확인은 점검 습관

데이터를 받자마자 duplicated().sum() — 짧지만 강력한 점검

01 · 0개

중복 없음

깨끗

추가 정리 필요 없음

02 · 0보다 큼

정리 필요

drop_duplicates
적용

정리 후 행이 줄었는지 검증

03 · 절반

심각 신호

같은 파일 두 번 합침
의심

수집·병합 과정 자체에 문제가 있을 가능성

새 물건의 흠집을 먼저 보는 것 같은 점검 습관

keep 옵션 이해하기

어느 것을 남기고 어느 것을 표시할지 결정

01 · 기본값

먼저 온 것을 살림

"first"

처음 행을 남기고 이후 중복 표시

대부분 이 값으로 충분

02 · 최신

나중 것을 살림

"last"

마지막 행을 남기고 이전 중복 표시

시간순 데이터에 적합

03 · 전체

모두 표시

False

겹친 행을 원본까지 모두 표시

어떤 행들이 겹쳤는지 한눈에 확인

시간순 갱신 데이터는 last로 최신을 남기고 옛 기록 삭제

keep 외우는 법

영어 단어 뜻이 곧 동작 — 한 번 연결하면 안 잊음

first는 먼저 온 것을 살림, last는 나중 것을 살림

이름과 동작이 일치

영어 단어 뜻이 곧 동작이라 한 번 연결하면 안 잊음

별도의 암기 부담이 없는 직관적 네이밍

대부분은 기본값 first로 충분

데이터 성격에 따라 last나 False가 필요할 때만 변경

처음엔 헷갈리지만 단어 그대로 기억하면 됨

posco

drop_duplicates()로 제거

한 줄로 중복 정리 — 18행 → 16행, 남은 중복 0

CODE

```
df_unique = df.drop_duplicates()
print(len(df), len(df_unique))
print(df_unique.duplicated().sum())
```

posco

제거 후 검증하기

전후 숫자를 맞춰 보는 검증이 실수를 막음

01 · 보존

원본 그대로 유지

새 변수
df_unique

결과는 새 변수에 담아 원본 보존

02 · 행 수

len 비교

중복 개수만큼
행이 줄었는지

중복이 2개였으니 행이 정확히 2개 줄어야 함

03 · 재확인

duplicated 합 = 0

남은 중복
0인지 확인

정리 후 duplicated 합이 0이면 정리 완료

처리 전후를 숫자로 맞춰 보는 검증 습관이 실수를 막음

posco

subset으로 기준 컬럼 지정

설비·회차가 같으면 다른 값이 달라도 중복으로 판정

CODE

```
df_u = df.drop_duplicates(subset=["machine_id", "cycle"], keep="last")
print(len(df_u))
```

keep="last"로 최신 기록 유지

subset 사용 원칙

무엇이 같으면 같은 기록인지 먼저 정하기

01 · 범위

컬럼 개수와 엄격함은 반비례

적게 넣을수록
많이 묶임

많이 넣을수록 엄격하게 중복 판정

02 · 정의

같은 기록 정의 먼저

무엇이 같으면
같은 기록인가

데이터 의미를 떠올리며 결정

03 · 주의

오판 위험

기준이 틀리면
멀쩡한 행 손실

기준을 잘못 잡으면 멀쩡한 행을 중복으로 오해

현장 규칙: 같은 설비·같은 시각 측정은 하나여야 함

posco

인덱스 정리 reset_index

중복 제거 후 0, 1, 2, 5, 7로 끊긴 인덱스를 0부터 다시 매김

CODE

```
df_clean = df.drop_duplicates().reset_index(drop=True)
print(df_clean.index)
```

drop=True 필수 · 16행이면 0~15까지

정제 한 바퀴 완성

이상치 정리 + 중복 정리 + 인덱스 정리 = 깨끗한 데이터 완성

drop=True로 옛 인덱스 버리고 0부터 새로 매김

drop 없이 실행하면 옛 인덱스가 새 컬럼으로 추가

행을 빼는 작업 뒤에는 reset_index로 인덱스를 맞추는 습관

이상치 제거, 중복 제거, 필터링 모두 동일

이상치도 정리하고 중복도 정리하고 인덱스까지 맞춘 데이터 완성

분석 결과를 바꾸지 않지만 다음 단계가 편해짐

깔끔하게 끝내는 습관이 다음 단계를 편하게 만듦

posco

03

직접 분석 실습

IQR 흐름과 중복 정리를 손으로 익히는 17개 실습

실습 1. IQR과 이상치 경계 구하기

사이클타임의 IQR과 1.5배 규칙 하한·상한 계산

목표

IQR과 1.5배 규칙으로 이상치 경계를 구하기

단계

· 사이클타임의 25%·75% 값을 구해 IQR(Q3-Q1) 계산

· Q1에서 IQR의 1.5배를 빼 하한 계산

· Q3에 IQR의 1.5배를 더해 상한 계산

예상 결과

사이클타임 IQR 15.12, 하한 -1.9·상한 58.6

posco

실습 1. IQR과 이상치 경계 구하기 정답

사이클타임의 IQR과 1.5배 규칙 하한·상한 계산

CODE · 실습 1

```
Q1 = df['사이클타임'].quantile(0.25)
Q3 = df['사이클타임'].quantile(0.75)
IQR = Q3 - Q1
print(round(Q1,2), round(Q3,2), round(IQR,2))   # 20.8 35.92 15.12
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
print(round(lower,2), round(upper,2))            # -1.89 58.61
```

실습 2. 조건 필터로 이상치 골라내고 개수·비율

경계 밖 값을 조건으로 골라 개수와 비율 확인

목표

경계 밖 값을 조건으로 골라 개수와 비율 확인

단계

· 하한보다 작거나 상한보다 큰 조건을 각각 괄호로 감싸 또는로 연결

· 조건에 맞는 이상치 행만 골라 확인

· sum으로 개수, mean으로 비율 계산

예상 결과

사이클타임 이상치 6건, 비율 3.0%

posco

실습 2. 조건 필터로 이상치 골라내고 개수·비율 정답

경계 밖 값을 조건으로 골라 개수와 비율 확인

CODE · 실습 2

```
mask = (df['사이클타임'] < lower) | (df['사이클타임'] > upper)
print(df[mask][['샷', '사이클타임', '상태']])
print(mask.sum(), round(mask.mean() * 100, 1))   # 6 3.0
```

실습 3. 박스플롯으로 이상치 확인

박스플롯 수염 밖 점이 조건으로 고른 값과 같은지 확인

목표

박스플롯으로 이상치를 그림으로 확인

단계

· 사이클타임 열을 박스플롯으로 그리기

· 수염 밖에 찍힌 점의 위치 확인

· 조건으로 고른 이상치와 그림의 점이 일치하는지 대조

예상 결과

수염 밖 점(6170 등 급증 샷)이 조건으로 고른 값과 일치

posco

실습 3. 박스플롯으로 이상치 확인 정답

박스플롯 수염 밖 점이 조건으로 고른 값과 같은지 확인

CODE · 실습 3

```
import matplotlib.pyplot as plt
df['사이클타임'].plot(kind='box')
plt.savefig('cycle_box.png'); plt.close()
# 수염 밖 점(6170 등)이 조건으로 고른 이상치와 같은지 확인
```

posco

실습 4. 이상치 제거 후 크기 비교

경계 밖 행을 빼고 남은 크기와 평균 확인

목표

경계 밖 행을 제거하고 남은 크기·평균 확인

단계

· 조건을 뒤집어 정상 범위 행만 남기기

· 원본과 제거 후의 행 수를 비교

· 제거 후 평균을 구해 변화 확인

예상 결과

202행 → 196행, 제거 후 사이클타임 평균 27.28

posco

실습 4. 이상치 제거 후 크기 비교 정답

경계 밖 행을 빼고 남은 크기와 평균 확인

CODE · 실습 4

```
정상 = df[~mask]
print(len(df), len(정상))                  # 202 196
print(round(정상['사이클타임'].mean(), 2))  # 27.28
```

posco

실습 5. 경계값 보정 clipping

이상치를 버리지 않고 경계값으로 눌러 보정

목표

이상치를 버리지 않고 경계값으로 눌러 보정

단계

· clip으로 하한보다 작은 값은 하한으로 올리기

· 상한보다 큰 값은 상한으로 내리기

· 보정 후 최솟값·최댓값·평균 확인

예상 결과

보정 후 최소 20.6·최대 58.6, 평균 28.28

posco

실습 5. 경계값 보정 clipping 정답

이상치를 버리지 않고 경계값으로 눌러 보정

CODE · 실습 5

```
보정 = df['사이클타임'].clip(lower=lower, upper=upper)
print(round(보정.min(), 2), round(보정.max(), 2))   # 20.6 58.61
print(round(보정.mean(), 2))                         # 28.28
```

실습 6. 처리 전후 통계 비교

제거·보정·중앙값 채움 세 처리의 평균 변화 비교

목표

제거·보정·중앙값 채움 세 처리의 평균 변화 비교

단계

· 실린더압력 이상치 경계와 조건을 만들기

· 제거·보정·중앙값 채움 세 방식을 각각 적용

· 처리 전 평균과 세 방식의 평균을 나란히 비교

예상 결과

전 234.31 → 제거 238.39·보정 235.31·채움 238.05

posco

실습 6. 처리 전후 통계 비교 정답

제거·보정·중앙값 채움 세 처리의 평균 변화 비교

CODE · 실습 6

```
Q1 = df['실린더압력'].quantile(0.25); Q3 = df['실린더압력'].quantile(0.75)
L = Q1 - 1.5*(Q3-Q1); U = Q3 + 1.5*(Q3-Q1)
m = (df['실린더압력'] < L) | (df['실린더압력'] > U)
채움 = df['실린더압력'].mask(m).fillna(df['실린더압력'].mask(m).median())
print(round(df['실린더압력'].mean(), 2))          # 234.31 (전)
print(round(df[~m]['실린더압력'].mean(), 2))      # 238.39 (제거)
print(round(df['실린더압력'].clip(L, U).mean(), 2))  # 235.31 (보정)
print(round(채움.mean(), 2))                       # 238.05 (채움)
```

posco

실습 7. duplicated로 중복 찾기와 개수

완전 중복 행을 찾고 keep 옵션으로 개수 비교

목표

완전 중복 행을 찾고 keep 옵션에 따른 개수 비교

단계

· duplicated로 중복 행 여부를 참·거짓으로 표시

· sum으로 중복 개수 세고 중복 행 직접 확인

· keep을 거짓으로 두면 겹친 행이 모두 표시되는 것 확인

예상 결과

완전 중복 2건, keep을 끄면 겹친 행 4건 표시

posco

실습 7. duplicated로 중복 찾기와 개수 정답

완전 중복 행을 찾고 keep 옵션으로 개수 비교

CODE · 실습 7

```
print(df.duplicated().sum())            # 2 (완전 중복)
print(df[df.duplicated()])              # 중복 행 확인
print(df.duplicated(keep=False).sum())  # 4 (겹친 행 모두)
```

posco

실습 8. drop_duplicates로 중복 제거

완전 중복 제거와 기준 컬럼 제거를 비교

목표

완전 중복 제거와 기준 컬럼 지정 제거를 비교

단계

· drop_duplicates로 완전 중복 행 제거

· 제거 후 행 수와 남은 중복 개수 확인

· subset으로 특정 컬럼만 기준 삼아 제거

예상 결과

202행 → 200행, 남은 중복 0, subset 기준도 200행

posco

실습 8. drop_duplicates로 중복 제거 정답

완전 중복 제거와 기준 컬럼 제거를 비교

CODE · 실습 8

```
유일 = df.drop_duplicates()
print(len(df), len(유일))          # 202 200
print(유일.duplicated().sum())      # 0
print(len(df.drop_duplicates(subset=['샷'], keep='last')))  # 200
```

posco

실습 9. reset_index로 인덱스 정리

중복 제거로 생긴 인덱스 구멍을 다시 매기기

목표

중복 제거로 생긴 인덱스 구멍을 0부터 다시 매기기

단계

· drop_duplicates로 중복을 제거

· reset_index로 인덱스를 0부터 다시 매기기

· 인덱스 최솟값·최댓값으로 연속성 확인

예상 결과

인덱스 0~199로 연속, 최종 200행

posco

실습 9. reset_index로 인덱스 정리 정답

중복 제거로 생긴 인덱스 구멍을 다시 매기기

CODE · 실습 9

```
정제 = df.drop_duplicates().reset_index(drop=True)
print(정제.index.min(), 정제.index.max())   # 0 199
print(len(정제))                             # 200
```

실습 10. 다른 현장(용접) 이상치·중복 종합 정제

탐색→보정→중복 점검→저장을 다른 현장에 그대로

목표

IQR 탐색부터 정제 데이터 저장까지 한 흐름으로

단계

· 용접 통전전류의 IQR 경계로 이상치 개수·비율 확인

· clip으로 이상치를 보정하고 중복을 제거·정리

· 정제한 데이터를 파일로 저장

예상 결과

용접 통전전류 이상치 24건(14.8%), 보정·중복 제거 후 저장

posco

실습 10. 다른 현장(용접) 이상치·중복 종합 정제 정답

탐색→보정→중복 점검→저장을 다른 현장에 그대로

CODE · 실습 10

```
wf = pd.read_csv('welding.csv')
c = '통전전류'
q1, q3 = wf[c].quantile(0.25), wf[c].quantile(0.75)
lo, hi = q1 - 1.5*(q3-q1), q3 + 1.5*(q3-q1)
m = (wf[c] < lo) | (wf[c] > hi)
print(int(m.sum()), round(m.mean()*100, 1))   # 24 14.8
wf[c] = wf[c].clip(lower=lo, upper=hi)
wf = wf.drop_duplicates().reset_index(drop=True)
wf.to_csv('cleaned_welding.csv', index=False)
print(len(wf))                                 # 158
```